SEARCHING FOR PAC-MAN

Intelligence Autonomous Agent course

APRIL 20, 2026

AUTHORS: MICHAEL O'SHEA (2025244317) MOHAMMED ABDELQADER (2025179643)

1. Introduction

This project focuses on improving decision-making in autonomous agents through the implementation of search algorithms in a Pac-Man environment. The main objective is to enhance ghost behavior using classical search strategies while maintaining their predefined personalities.

According to the project statement, the task involves replacing simplistic movement logic with intelligent pathfinding using algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Greedy Search, and A*. These algorithms allow agents to compute paths in a transition graph representing the environment.

From a theoretical perspective, this aligns with decision-making under uncertainty, where an agent selects actions based on expected outcomes and utility. The project bridges search (deterministic decision-making) and decision theory (complex decision-making).

2. Experimental Setup

The experimental framework is implemented in pacman.py, where multiple simulations are executed to evaluate performance.

- **Number of experiments:** 100 runs per algorithm
- **Algorithms tested:** BFS, DFS, Greedy and A*
- **Evaluation metrics:** Average score, wins, success rate, mean path lengths, no path found rate, stationary and Agent moving rate
- Each ghost uses a specific strategy:
  Blinky : Directly chases Pac-Man
  Pinky : Attempts interception
  Inky : Randomized goal
  Clyde : Hybrid (chase or random)

These behaviors are preserved while integrating search-based navigation.

3. Implementation

Note: Comprehensive comments have been included throughout the code to facilitate understanding of the implementation of each algorithm. Therefore, a detailed written description is not provided.

3.1 Search Algorithms

3.1.1 Breadth-First Search (BFS)

The BFS implementation was based on the theoretical principles discussed in class, using a queue (FIFO) to manage node expansion. The algorithm applied in this project adheres precisely to the approach presented in the course material.

```python
def breadth_first_search(start, goal, transition_graph):
    # Breadth-first search returning the sequence of positions from start to goal,
    # excluding the start position so move_in_path can consume the next step.
    #"print("BFS: Computing path from", start, "to", goal)
    if start == goal:
        return []

    # first initialize the queue with the start position, and a visited set to keep track of visited nodes,
    # and a parent dictionary to reconstruct the path once we reach the goal
    queue = deque([start])
    visited = {start}
    parent = {start: None}

    while queue:
        # pop the first position from the queue and check if it is the goal, if it is then reconstruct the path using the parent dictionary and return it
        current = queue.popleft()
        # if it is not the goal, then add its unvisited neighbours to the queue and mark them as visited, and set their parent to the current position
        for neighbour in transition_graph.get(current, []):
            # if the neighbour has already been visited, then skip it to avoid infinite loops and redundant processing
            if neighbour in visited:
                continue

            # now add the neighbour to the visited set and the queue, and set its parent to the current position
            visited.add(neighbour)
            parent[neighbour] = current

            # if the neighbour is the goal, then reconstruct the path using the parent dictionary and return it. The path should be a list of
            # positions from start to goal, excluding the start position so move_in_path can consume the next step.
            if neighbour == goal:
                path = []
                node = goal
                # reconstruct the path by following the parent pointers from the goal back to the start, and then reverse it to get the path from start to goal
                while node != start:
                    path.append(node)
                    node = parent[node]
                # reverse the path to get the correct order from start to goal, and return it
                path.reverse()
                return path

            queue.append(neighbour)

    return None
```

3.1.2 Depth-First Search (DFS)

The DFS implementation was based on the theoretical principles discussed in class, using a stack (LIFO) to manage node expansion. The algorithm applied in this project again adheres precisely to the approach presented in the course material.

```python
def depth_first_search(start, goal, transition_graph):
    #TODO: Implement the depth-first search algorithm. It should return the path as a list of positions
    # Hint: you can use a stack (LIFO) data structure to implement the DFS. You can use a list and append/pop
    #  from the end to implement the stack. You should also keep track of visited nodes to avoid infinite loops.

    #print("DFS: Computing path from", start, "to", goal)
    if start == goal:
        return []

    # first initialize the stack with the start position, and a visited set to keep track of visited nodes,
    stack = [start]
    visited = {start}
    parent = {start: None}

    #print("DFS: entering stack", start, "to", goal)
    while stack:
        # pop the last position from the stack and check if it is the goal, if it is then reconstruct the path using the parent dictionary and return it
        current = stack.pop()
        # if it is not the goal, then add its unvisited neighbours to the stack and mark them as visited, and set their parent to the current position
        for neighbour in transition_graph.get(current, []):
            if neighbour in visited:
                continue

            # check if neighbour is already in stack, if it is then we don't add it again, to
            # avoid duplicates in the stack which can lead to longer paths and more time spent in the search
            #if neighbour in stack:
            #    print("DFS: neighbour", neighbour, "already in stack, skipping")
            #    continue

            visited.add(neighbour)
            parent[neighbour] = current

            # if the neighbour is the goal, then reconstruct the path using the parent dictionary and return it. The path should be a list of
            if neighbour == goal:
                path = []
                node = goal
                # reconstruct the path by following the parent pointers from the goal back to the start, and then reverse it to get the path from start to goal
                while node != start:
                    path.append(node)
                    node = parent[node]
                path.reverse()
                #print("DFS: path found from", start, "to", goal, ":", path)
                return path

            stack.append(neighbour)

    #print("DFS: No path found from", start, "to", goal)
    return None
```

3.1.3 Greedy

The Greedy search algorithm was implemented according to the concepts taught in class, using a priority queue to control the expansion of nodes. The priority values were computed using a heuristic function, specifically the Manhattan distance from the current node to the goal, ensuring consistency with the theoretical approach presented in the course material.

```python
def greedy_search(start, goal, transition_graph):
    #TODO: Implement the greedy search algorithm. It should return the path as a list of positions
    # use a priority queue data structure to implement the greedy search.
    # use the heapq library in Python to implement the priority queue.
    # The priority should be based on the heuristic function, which in this case can be the Manhattan distance
    # from the current node to the goal. Also keep track of visited nodes to avoid infinite loops.
    if start == goal:
        return []

    import heapq
    # the heap will store tuples of (priority, position), where priority is the Manhattan distance from the position to the goal. The heap is initialized with the
    # start position and its priority, which is the Manhattan distance from the start
    heap = [(game_engine.manhattan_distance(start, goal), start)]
    visited = {start}
    parent = {start: None}

    while heap:
        # pop the position with the lowest priority (the one estimated to be closest to the goal based on the heuristic function) from the heap
        # and check if it is the goal, if it is then reconstruct the path using the parent dictionary and return it
        _, current = heapq.heappop(heap)
        for neighbour in transition_graph.get(current, []):
            if neighbour in visited:
                continue

            visited.add(neighbour)
            parent[neighbour] = current

            # if the neighbour is the goal, then reconstruct the path using the parent dictionary and return it. The path should be a list of positions
            # from start to goal, excluding the start position so move_in_path can consume the next step.
            if neighbour == goal:
                path = []
                node = goal
                while node != start:
                    path.append(node)
                    node = parent[node]
                # reverse the path to get the correct order from start to goal, and return it
                path.reverse()
                return path
            # the priority is based on the heuristic function, which in this case can be the Manhattan distance from the current node to the goal. the prioirity queue is sorted
            #  by the priority, so the node with the lowest priority will be popped first, which is the node that is estimated to be closest to the goal based on the
            # heuristic function. In case of ties in the priority, the node that was added to the heap first will be popped first, which can lead to different
            # paths being explored and can affect the performance of the algorithm.
            heapq.heappush(heap, (game_engine.manhattan_distance(neighbour, goal), neighbour))

    return None
```

3.1.4 A*

The A* search algorithm was developed in accordance with the principles discussed in class, utilizing a priority queue to control node expansion. The priority values were calculated using the function f(k) = g(k) + h(k), where g(k) represents the number of steps from the start node to the current node, and h(k) is the heuristic estimate given by the Manhattan distance to the goal. This ensures full consistency with the theoretical formulation taught in the course.

```python
def a_star_search(start, goal, transition_graph):
    #TODO: Implement the A* search algorithm. It should return the path as a list of positions
    # use a priority queue data structure to implement the A* search.
    # use the heapq library in Python to implement the priority queue.
    # The priority should be based on the cost function, which is the sum of the path cost from the start node to the
    # current node and the heuristic function, which in this case will be the Manhattan distance from the current node to the goal.
    # Also keep track of visited nodes to avoid infinite loops.
    if start == goal:
        return []

    import heapq

    # the heap will store tuples of (priority, cost, position), where priority is the sum of the path cost from the start node to the current node and the
    #  heuristic function, which in this case will be the Manhattan distance from the current node to the goal. The heap is initialized with the start position,
    # its priority, which is the Manhattan distance from the start, and its cost, which is 0.
    heap = [(0 + game_engine.manhattan_distance(start, goal), 0, start)]
    visited = {start: 0}
    parent = {start: None}

    while heap:
        # pop the position with the lowest priority (the one estimated to be closest to the goal based on the heuristic function and the path cost) from the heap
        # and check if it is the goal, if it is then reconstruct the path using the parent dictionary and return it .
        _, cost, current = heapq.heappop(heap)
        if current == goal:
            path = []
            node = goal
            while node != start:
                path.append(node)
                node = parent[node]
            # reverse the path to get the correct order from start to goal, and return it
            path.reverse()
            return path

        for neighbour in transition_graph.get(current, []):
            new_cost = cost + 1  # assuming uniform cost for each step
            if neighbour in visited and visited[neighbour] <= new_cost:
                continue

            visited[neighbour] = new_cost
            parent[neighbour] = current
            # the priority is the sum of the path cost from the start node to the current node and the heuristic function, which in this case will be the Manhattan distance
            # from the current node to the goal. the prioirity queue is sorted by the priority, so the node with the lowest priority will be popped first,
            # which is the node that is estimated to be closest to the goal based on the heuristic function and the path cost. In case of ties in the priority,
            # the node that was added to the heap first will be popped first, which can lead to different paths being explored and can affect the performance of the algorithm.
            heapq.heappush(heap, (new_cost + game_engine.manhattan_distance(neighbour, goal), new_cost, neighbour))

    return None
```

3.2 Implementing the Ghosts

3.2.1 Blinky → goal = Pac-Man position

```python
def blinky_search_agent(search_algorithm):
    def agent(ghost, game_state):
        #TODO: Implement the search agent for Blinky
        #1.Define the start and goal positions (as tuples of x,y)
        start = (ghost['x'], ghost['y'])
        pacman = game_state['pacman']
        goal = (pacman['x'], pacman['y'])
        #print("Blinky: Computing path from", start, "to", goal)

        #2.Use the compute_path function to compute the path from start to goal. The transition graph is already computed and is accessible in game_state['transition_graph']
        path = compute_path(start, goal, game_state['transition_graph'], search_algorithm, ghost)
        #print("Blinky: Path found from", start, "to", goal, ":", path)
        # store goal and path in ghost's state for debugging and analysis purposes
        ghost['current_goal'] = goal
        ghost['current_path'] = path
        #3.Use the move_in_path function to move the ghost in the first step of the path
        move_in_path(ghost, game_state, path, search_algorithm)

        current_pos = (ghost['x'], ghost['y'])
        #print("Blinky: Current position after moving in path:", current_pos)
    return agent
```

3.2.2 Pinky → goal = predicted position ahead of Pac-Man

```python
def pinky_search_agent(search_algorithm):
    def agent(ghost, game_state):
        #1.Define the start and goal positions (as tuples of x,y)
        start = (ghost['x'], ghost['y'])
        pacman = game_state['pacman']
        # find all positions in the transition graph that are within 4 steps of pacman, and choose the one that is
        # closest to pinky as the goal. This will make pinky try to cut off pacman by targeting a position in front of him,
        # rather than directly targeting pacman's current position like blinky.
        all_positions = list(game_state['transition_graph'].keys())
        positions_with_distance = [(pos, game_engine.manhattan_distance(pos, (pacman['x'], pacman['y']))) for pos in all_positions]
        positions_with_distance = [pd for pd in positions_with_distance if pd[1] <= 4]

        goal = min(positions_with_distance, key=lambda pd: game_engine.manhattan_distance(pd[0], start))[0]

        #goal = (pacman['x'], pacman['y'])
        #print("Pinky: Computing path from", start, "to", goal)

        #2.Use the compute_path function to compute the path from start to goal. The transition graph is already computed and is accessible in game_state['transition_graph']
        path = compute_path(start, goal, game_state['transition_graph'], search_algorithm, ghost)

        # store goal and path in ghost's state for debugging and analysis purposes
        ghost['current_goal'] = goal
        ghost['current_path'] = path
        #3.Use the move_in_path function to move the ghost in the first step of the path
        move_in_path(ghost, game_state, path, search_algorithm)
    return agent
```

3.2.3 Inky → goal = random node

```python
def inky_search_agent(search_algorithm):
    def agent(ghost, game_state):
        #TODO: Implement the search agent for Inky
        #1.Define the start and goal positions (as tuples of x,y)
        start = (ghost['x'], ghost['y'])
        # Goal will be a random position in the transition graph, to add some randomness to Inky's behaviour and differentiate it from Blinky and Pinky,
        # which will always target pacman's current position. This is a simple way to add some unpredictability to Inky's behaviour,
        # which can be turned on or off by changing the definition of the goal.
        # if goal is equal to start then pick a new random

        current_goal = ghost.get('current_goal')
        if current_goal is not None and current_goal != start:
            goal = current_goal
        else:
            goal = random.choice(list(game_state['transition_graph'].keys()))

        #print("Inky: Computing path from", start, "to", goal)

        #2.Use the compute_path function to compute the path from start to goal. The transition graph is already computed and is accessible in game_state['transition_graph']
        path = compute_path(start, goal, game_state['transition_graph'], search_algorithm, ghost)

        # store goal and path in ghost's state for debugging and analysis purposes
        ghost['current_goal'] = goal
        ghost['current_path'] = path
        #3.Use the move_in_path function to move the ghost in the first step of the path
        move_in_path(ghost, game_state, path, search_algorithm)
    return agent
```

3.2.4 Clyde → conditional goal (chase or random)

```python
def clyde_search_agent(search_algorithm):
    def agent(ghost, game_state):
        #TODO: Implement the search agent for Clyde
        #1.Define the start and goal positions (as tuples of x,y)
        start = (ghost['x'], ghost['y'])
        pacman = game_state['pacman']
        # if distance to pacman is greater than 5 then goal is pacman's position, else goal
        # is a random position in the transition graph, to add some randomness to Clyde's behaviour and differentiate it from the other ghosts,
        # which will always target pacman's current position. This is a simple way to add some unpredictability to Clyde's behaviour,
        # which can be turned on or off by changing the definition of the goal.
        distance_to_pacman = game_engine.manhattan_distance(start, (pacman['x'], pacman['y']))
        if distance_to_pacman > 5:
            goal = (pacman['x'], pacman['y'])
        else:
            goal = random.choice(list(game_state['transition_graph'].keys()))

        #print("Clyde: Computing path from", start, "to", goal)

        #2.Use the compute_path function to compute the path from start to goal. The transition graph is already computed and is accessible in game_state['transition_graph']
        path = compute_path(start, goal, game_state['transition_graph'], search_algorithm, ghost)
        #3.Use the move_in_path function to move the ghost in the first step of the path
        move_in_path(ghost, game_state, path, search_algorithm)
        # store goal and path in ghost's state for debugging and analysis purposes
        ghost['current_goal'] = goal
        ghost['current_path'] = path
    return agent
```

For all ghosts other than Inky, the path is recomputed at each time step, starting from the ghost's current position toward Pac-Man's current location. This is required because Pac-Man is not stationary, and his movement causes previously computed paths to become obsolete. In contrast, Inky operates with a random movement strategy, so its path is only recalculated when it reaches its current goal rather than at every step.

3.2.5 Run away from Pac-Man

```python
def run_away_from_pacman_search(search_algorithm):
    def agent(ghost, game_state):
        #TODO: Implement the search agent for running away from pacman (used by all the ghosts in the frightened state)

        #1.Define the start and goal positions (as tuples of x,y
        # start is the current position of the ghost
        start = (ghost['x'], ghost['y'])
        # Goal will be further distance from pacman, so we can define it as the position in the transition graph
        # that is furthest from pacman, which we can find by iterating over all the positions in the transition graph
        # and computing their distance to pacman, and taking the one with the maximum distance. This is a simple way to
        # define a goal for running away from pacman.
        pacman = game_state['pacman']
        max_distance = -1
        goal = start
        for pos in game_state['transition_graph'].keys():
            distance = pacman_perceptions.pacman_distance_to_position(game_state, pos)
            if distance > max_distance:
                max_distance = distance
                goal = pos
        #print("Run away: Computing path from", start, "to", goal)
        #2.Use the compute_path function to compute the path from start to goal. The transition graph is already computed and is accessible in game_state['transition_graph']

        path = compute_path(start, goal, game_state['transition_graph'], search_algorithm, ghost)
        #3.Use the move_in_path function to move the ghost in the first step of the path
        move_in_path(ghost, game_state, path, search_algorithm)

    return agent
```

When in the frightened state, all ghosts utilize the run-away-from-Pac-Man search strategy, selecting paths that maximize their distance from Pac-Man based on the Manhattan distance. This approach can result in stationary behavior, as the maximum distance may remain unchanged between consecutive steps.

4. Challenges Faced

4.1 DFS Loops and Instability

DFS was completely the wrong search algorithm for this problem, regardless of the scores that can be seen in sections 5 below DFS caused a number of issues:

- Cyclic paths
- Back-and-forth movement
- Poor chasing behavior

This is because DFS:

- Does not guarantee shortest paths
- Is highly dependent on node ordering
- We recomputed paths every frame

Therefore, DFS is not well-suited for moving-target problems due to its unstable behavior and the need for frequent re-planning. To mitigate these issues and enable the completion of 100 runs (which was not possible with the unmodified implementation), adjustments were made to the **move_in_path** function specifically for DFS. In particular, opposite directions were restricted, even when they would move the agent closer to the goal. This modification reduced oscillatory behavior and prevented the ghost from becoming trapped in simple loops; however, more substantial changes would be required to fully eliminate all cyclic behavior.

4.2 Moving Target Problem

Since Pac-Man is constantly moving, computing a path only once is insufficient, as the target position changes before the ghost can reach it. As a result, the path must be recalculated at every step (with the exception of Inky). This continuous re-planning increases computational overhead, particularly for inefficient algorithms such as DFS, and plays a key role in determining which search algorithm is most appropriate for the Pac-Man environment.

5. Results and Analysis

5.1 Experimental Results

Search Algorithm: BFS				
Ghost Name	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
Blinky	15.151	0	0.058	0.942
Pinky	13.599	0	0.150	0.850
Inky	15.689	0	0.121	0.879
Clyde	17.536	0	0.065	0.934
Average score of 100 experiments: 193.91 Wins: 1 Success rate: 1%				
Search Algorithm: DFS				
Ghost Name	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
Blinky	85.436	0	3.134e-05	0.999
Pinky	74.268	0	0.030	0.969
Inky	98.147	0	0.000	1
Clyde	87.321	0	9.962e-05	0.999
Average score of 100 experiments: 303.16 Wins: 47 Success rate: 47%				
Search Algorithm: Greedy				
Ghost Name	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
Blinky	17.268	0	0.148	0.852
Pinky	15.165	0	0.190	0.810
Inky	17.434	0	0.164	0.836

Clyde	19.258	0	0.148	0.852
Average score of 100 experiments: 199.63 Wins: 4 Success rate: 4%				
Search Algorithm: A*				
Ghost Name	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
Blinky	15.376	0	0.129	0.871
Pinky	13.820	0	0.187	0.813
Inky	15.382	0	0.172	0.828
Clyde	17.176	0	0.129	0.871
Average score of 100 experiments: 202.37 Wins: 2 Success rate: 2%				

Summary of results			
Search Algorithm	**Average score**	**Wins**	**Success rate**
BFS	193.91	1	1%
DFS	303.16	47	47%
Greedy	199.63	4	4%
A*	202.37	2	2%

Ghost name: Blinky				
Search Algorithm	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
BFS	15.151	0	0.058	0.942
DFS	85.436	0	3.134e-05	0.999
Greedy	17.268	0	0.148	0.852
A*	15.376	0	0.129	0.871
Ghost name: Pinky				
Search Algorithm	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
BFS	13.599	0	0.150	0.850
DFS	74.268	0	0.030	0.969
greedy	15.165	0	0.190	0.810
A*	13.820	0	0.187	0.813
Ghost name: Inky				
Search Algorithm	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
BFS	15.689	0	0.121	0.879
DFS	98.147	0	0.000	1
Greedy	17.434	0	0.164	0.836
A*	15.382	0	0.172	0.828

Ghost name: Clyde				
Search algorithm	**Mean path lengths**	**No path found rate**	**Stationary rate**	**Moving rate**
BFS	17.536	0	0.065	0.934
DFS	87.321	0	9.962e-05	0.999
Greedy	19.258	0	0.148	0.852
A*	17.176	0	0.129	0.871

5.2 Performance Comparison

Search Algorithm	Path Quality	Speed	Stability	Overall Performance
DFS	Poor	Medium	Low	Worst
BFS	Optimal	Slow	High	Good
Greedy	Approximate	Fast	Medium	Moderate
A*	Optimal	Fast	High	Best

7. Conclusion

This project demonstrates the application of search algorithms in autonomous decision-making within a dynamic environment. The integration of BFS, DFS, Greedy, and A* enabled a comparative analysis of their effectiveness in real-time pursuit tasks.

The results show that:

- DFS is unsuitable for dynamic environments due to instability and inefficiency
- BFS guarantees optimal paths but is computationally expensive
- Greedy provides fast but suboptimal decisions
- **A*** offers the best balance between efficiency and performance

Ultimately, A* emerges as the most suitable algorithm for this application due to its ability to combine heuristic guidance with optimal pathfinding, making it ideal for complex decision-making scenarios in autonomous agents.

Remark: ChatGPT assisted only with report wording and played no role in the core analysis or results.